Week 4: Tree recursion, BFS, and BSTs

Depth and ordering in hierarchical records

A task tree has at most two subtasks per node. What is its deepest level? We also need a report grouped by level and a check that a search tree preserves its ordering rules.

Each node can branch in two directions. Recursive return values, stacks, queues, and ancestor bounds carry the information needed for these questions.

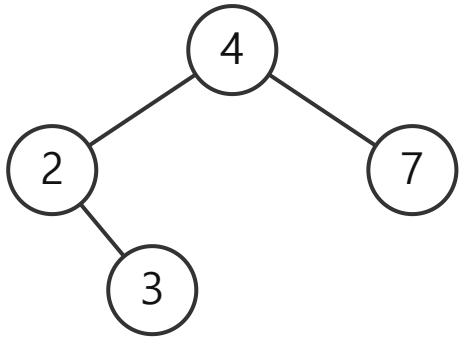
Week 4 problems and learning outcomes

Session 7: recursive returns and maximum depth. Session 8: BFS, BST validation, and inorder traversal.

- Define empty-tree and child-call return values.
- Track level boundaries and ancestor constraints.
- Separate time, auxiliary space, and output space.
- Understand recursion on small trees; use iteration for long chains.

Practice: LeetCode 104, 102, 98, 230. Diagrams, traces, and midpoint checks are additional teaching examples.

Binary-tree nodes and input notation



`[4,2,7,None,3]`: level-order notation. `None` marks a missing child; subsequent entries fill the left and right children of queued parents.

Depth counts nodes from the root: 0 for an empty tree, 1 for a root alone.

TreeNode and local variables

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right
```

- `left`, `right`: references to child nodes
- Separate local variables for each recursive call
- `return`: pass a result back to the caller

Combine the returned child heights to obtain the current height.

104 · Problem and state

LeetCode 104: Maximum Depth of Binary Tree · Easy

Return the maximum root-to-leaf path length measured in nodes.

`root=[4,2,7,None,3] → 3`

Derive depth(None)=0 and depth(node)=1+max(left,right) on small trees; the iterative implementation uses an explicit (node,depth) stack.

Recursive depth computation

```
def depth_recursive(root):
    """Small-tree demonstration; Python's recursion limit applies."""
    if root is None:
        return 0
    return 1 + max(depth_recursive(root.left), depth_recursive(root.right))
```

The empty tree has height 0. Add the current node to the larger child height. This function is for small-tree demonstrations.

Subtree return values

call	left height	right height	return
depth(3)	0	0	1
depth(2)	0	1	2
depth(7)	0	0	1
depth(4)	2	1	3

The parent waits for both child results before combining them.

Explicit stack for maximum depth

```
def max_depth(root):
    """Explicit stack supports skew trees within the official size limit."""
    if root is None:
        return 0
    stack, best = [(root, 1)], 0
    while stack:
        node, depth = stack.pop()
        best = max(best, depth)
        if node.left:
            stack.append((node.left, depth + 1))
        if node.right:
            stack.append((node.right, depth + 1))
    return best
```

104 · Correctness and cost

Structural induction proves the recurrence. The explicit stack carries the true root depth to every node and records its maximum.

n is node count, h height. O(n) time, O(h) DFS space, O(n) worst case.

Skipped branches and recursion depth

`if left ... elif right ...` skips the right child when the left child exists. Check both children with separate `if` statements.

A chain can exceed Python's recursion limit. Use the explicit-stack implementation for the official maximum of 10,000 nodes.

Checks: empty tree, one node, and a long right-only chain.

102 · Problem and state

LeetCode 102: Binary Tree Level Order Traversal · Medium

Return tree values grouped by depth, left to right.

`root=[4,2,7,None,3] → [[4],[2,7],[3]]`

Pop exactly the initial len(queue) nodes, appending children for the next level.

deque and level boundaries

```
from collections import deque
queue = deque([root])
```

- `popleft()`: remove the earliest queued node
- `append(child)`: enqueue a next-level child
- `range(len(queue))`: fix the node count at level entry

Children added during processing are visited in the next outer iteration.

Queue trace by level

Queue at entry	Nodes to remove	Output level	Next queue
[4]	1	[4]	[2,7]
[2,7]	2	[2,7]	[3]
[3]	1	[3]	[]

After removing the current level, only the next level remains. Enqueue left children before right children.

BFS level-order implementation

```
def level_order(root):
    if root is None:
        return []
    queue, result = deque([root]), []
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

Import `deque` from `collections`.

102 · Correctness and cost

At each outer-loop entry the queue contains one level. Processing its initial size leaves only the next level; left-first insertion preserves order.

n is node count, w maximum width. O(n) time, O(w) queue space, O(n) output.

Queue operations and result storage

`list.pop(0)` shifts the remaining entries and can lead to $O(n^2)$ total work on wide trees. Removal with `deque.popleft()` costs $O(1)$.

The queue can hold the remainder of one level and part of the next. Each level has at most w nodes, so queue storage is $O(w)$.

The returned level lists separately occupy $O(n)$ space.

98 · Problem and state

LeetCode 98: Validate Binary Search Tree · Medium

Decide whether all descendants satisfy strict BST ordering.

`root=[8,3,10,None,None,6,12] → False`

Carry an open interval (lower,upper); tighten the upper bound on the left and lower bound on the right.

Open bounds inherited from ancestors

- Root: `(-inf, inf)`
- Left child: `(lower, node.val)`
- Right child: `(node.val, upper)`
- Check: `lower < node.val < upper`

Strict inequalities reject duplicate keys. Every descendant in the right subtree of root 8 must exceed 8.

Ancestor-bound counterexample

node	lower	upper	decision
8	-inf	inf	pass
10	8	inf	pass
6	8	10	6<=8, fails

Selected nodes on the root-to-6 path; other visited nodes are omitted. The value 6 passes the parent comparison with 10 but fails the inherited lower bound 8.

BST validation implementation

```
def valid_bst(root):
    stack = [(root, float("-inf"), float("inf"))]
    while stack:
        node, lower, upper = stack.pop()
        if node is None:
            continue
        if not lower < node.val < upper:
            return False
        stack.append((node.right, node.val, upper))
        stack.append((node.left, lower, node.val))
    return True
```

98 · Correctness and cost

The interval is the intersection of all ancestor constraints. Tightening it preserves that meaning; any violation invalidates the whole tree.

n is node count, h height. O(n) time, O(h) explicit DFS space, O(n) worst case.

230 · Problem and state

LeetCode 230: Kth Smallest Element in a BST · Medium

Return the kth smallest BST value.

`root=[5,2,8,1,3], k=3 → 3`

Use an explicit inorder stack; decrement k when visiting a node.

Inorder traversal and visit order

For BST `[5,2,8,1,3]`, inorder values are `1,2,3,5,8`.

Visited value	Remaining k	Action for initial k=3
1	2	continue
2	1	continue
3	0	return 3

Store nodes while descending left. Visit a popped node, then enter its right subtree.

Kth-smallest implementation

```
def kth_smallest(root, k):
    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        k -= 1
        if k == 0:
            return node.val
        node = node.right
    raise ValueError("k exceeds tree size")
```

230 · Correctness and cost

Every left value is smaller and every right value larger. Induction makes inorder sorted, so the kth visit gives the answer.

n is node count, h height. O(h+k) time, O(n) worst case, O(h) space.

Baseline methods and costs

Task	Baseline	Current implementation
Maximum depth	recursion on small trees	explicit DFS stack
Level order	remove from front of list	O(1) deque removal
BST validation	recheck descendants: $O(n^2)$ worst case	carry bounds: O(n)
Kth value	store full inorder: O(n) time/space	stop early: O(h+k) time, O(h) space

Height h can reach n in a chain. Integer comparisons and node references are treated as constant-cost operations.

Midpoint checks and submission

1. Predict the depth and levels of `[4,2,7,None,3]` before execution.
2. Decide whether `[8,3,10,None,None,6,12]` is a BST and justify your answer.
3. Find the outputs for k=1, 3, 5 in BST `[5,2,8,1,3]`.

Submit code, state definitions, correctness, costs, and boundary cases. Attempt the student exercises before consulting the companion.

Execution and sources

Extract the Week 4 code and run these commands in the `code` directory.

```
python demo_07.py
python demo_08.py
python verify.py
python verify.py --student
```

Run the last command after implementing `student_week_04.py`.

Erik Demaine and Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).